

Assignment FEM21045

September, 2026

Welcome to the assignment of the Machine Learning in Econometrics course! This assignment should be made in **groups of four** students, so please team up with your fellow students. The setup of this assignment is somewhat non-standard, so please read this document carefully.

In this assignment, you are asked to build neural-network models that predict the probability that an observation defaults. The purpose of the assignment is not only to obtain accurate probability predictions, but also to study how the objective used to train a neural network changes its fitted predictions when prediction errors have asymmetric economic consequences.

1 Overarching Task

Your overarching task is to train neural networks for a binary prediction problem. You will be provided with a preprocessed and anonymized dataset. Each observation consists of a binary target and a set of predictor variables. The target takes the value 1 if the observation defaults and 0 otherwise.

You are provided with three datasets on Canvas:

- `X_trn.csv`, containing the predictor variables for the training set;
- `y_trn.csv`, containing the corresponding binary default indicators; and
- `X_test.csv`, containing the predictor variables for the test set, without the target.

The supplied features have already been preprocessed so that you can train a neural network without extensive additional data manipulation. You may perform further preprocessing or feature engineering, but every transformation must be estimated using the training data only and then applied unchanged to validation and test observations.

In designing your predictive models, please adhere to the following requirements:

1. **Your final models must be neural networks.** Non-neural-network methods may be used as informal benchmarks during development, but they cannot be submitted as either of the two final models. Ensembles that combine a neural network with other model classes are not allowed.
2. You must train **two final neural-network models**.
3. The first model should be trained to minimize the **Brier score**:

$$\text{BS} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{p}_i)^2,$$

where $\hat{p}_i \in [0, 1]$ is the predicted probability that $y_i = 1$, and $y_i \in \{0, 1\}$ is the observed outcome. Lower scores indicate better predictions, with a score of zero corresponding to perfect predictions.

4. The second model should be trained using an **asymmetric economic loss**: prediction errors for observations that actually default receive three times the weight of prediction errors for observations that do not default:

$$L_{\text{econ}} = \frac{1}{n} \sum_{i=1}^n w_i (y_i - \hat{p}_i)^2, \quad w_i = \begin{cases} 3, & \text{if } y_i = 1, \\ 1, & \text{if } y_i = 0. \end{cases}$$

5. Your submitted predictions must be **probabilities**, not binary classifications.
6. When estimating and tuning your models, you should optimize hyperparameters and evaluate predictive performance using an appropriate training and validation strategy. The exact strategy and the hyperparameters you tune are up to you. The test targets remain hidden and `X_test.csv` may not be used to select models, hyperparameters, preprocessing choices or stopping rules.

1.1 Implementation requirements

You must write the code for both neural networks yourselves in Python or R. The **forward pass, loss functions, gradients and parameter updates must be implemented using plain array or matrix arithmetic**. You may use the numerical routines explicitly permitted below, but you may not use ready-made neural network implementations, automatic differentiation, or optimization routines to implement these components. Any package may be used for loading, cleaning, splitting, plotting, evaluating and profiling, subject to the exclusions below.

Python

Allowed:

- `numpy`, including `numpy.random` and `numpy.linalg`.
- `scipy.special` for `expit`, `logsumexp` and `softmax`, and `scipy.linalg`.
- `pandas` or `polars` for data handling.
- `matplotlib` or `seaborn` for plotting.
- `cProfile` with `snakeviz`, complemented by `line_profiler`. The function-level view in `snakeviz` can be complemented by line-level profiling, which is particularly useful when a vectorized multilayer perceptron consists of only a few functions.

Forbidden: `torch`, `tensorflow`, `keras`, `jax`, `autograd`, `flax`, `sklearn.neural_network`, `scipy.optimize`, `numba`, `cython` and `cupy`.

R

Allowed:

- Base matrix operations, `crossprod`, `tcrossprod` and `sweep`, and the `Matrix` package for sparse inputs.
- `data.table`, `dplyr`, `readr` and `tidyr` for data handling.
- `ggplot2` for plotting.
- `profvis` for profiling, with `bench` or `microbenchmark` for timing.

Forbidden: `torch`, `keras`, `tensorflow`, `nnet`, `neuralnet`, `RSNNS`, `h2o`, `mlr3` learners, `parsnip::mlp`, `stats::optim`, `nlm`, `optimx`, `Deriv` and any other automatic-differentiation package, `Rcpp` and `RcppArmadillo`.

If you are unsure whether another package or function is allowed, contact the TA before using it!

2 Graded Subtasks (20 pt)

You are graded on two broad components: predictive performance in the prediction challenge and your ability to motivate, implement and discuss your modeling choices concisely.

2.1 Prediction Challenge (5 pt)

The prediction challenge evaluates the two final models on the held-out test set. The 5 points allocated to predictive performance are split equally across the two objectives:

- the score achieved by the model trained to minimize the Brier score
- the score achieved by the model trained to minimize the asymmetric economic loss.

For each objective, your performance is compared with benchmark predictions constructed by the TA and the base rate of default. **If you beat the TA’s benchmark, you get full points!**

Beating the base rate of default corresponds to 1 point, and beating the TA benchmark corresponds to 10 points. Grades for performance between these benchmarks are determined by linear interpolation:

$$\text{Grade} = 1 + 9 \frac{s - s_{\text{Baserate}}}{s_{\text{TA}} - s_{\text{Baserate}}},$$

where s denotes the student’s performance, and s_{Baserate} and s_{TA} denote the performance of the base rate and TA benchmarks, respectively.

Prediction file

Submit the two sets of test-set probabilities in a single NumPy file called `predictions.npy`. It must be a two-dimensional array of shape

$$(n_{\text{test}}, 2).$$

The first column must contain the probabilities from the Brier-score model and the second column the probabilities from the economic-loss model, in exactly the same observation order as `X_test.csv`. In Python, for example, you can stack the two probability vectors in this order and save the resulting array using `np.save`.

Your prediction file will be evaluated automatically. Please check that the file name, shape, row ordering and column ordering are correct and that every entry is finite and lies in $[0, 1]$.

Please note that you are **not allowed to use the hidden test outcomes in any way during model development**. Any attempt to use it for training, fitting preprocessing transformations, model selection, or tuning is considered fraud!

You must also submit the complete replication code. The code is not graded directly, but after the deadline you may be asked to reproduce your submitted predictions using the submitted code.

2.2 Report (15 pt)

Your second subtask is to report on your design choices, methodology and main findings. The report should be concise but sufficiently precise that a peer following the course could reproduce your procedure up to stochastic training variation.

Your report should address the following points:

1. Describe your preprocessing, network architecture, optimization procedure, training-validation strategy and hyperparameter search. Statements such as “we used a grid search” are not sufficient unless you report which hyperparameters and candidate values were considered.

2. Report the out-of-sample validation performance of **both networks under both loss functions**. A compact 2×2 comparison is sufficient. This allows you to show the trade-off induced by changing the training objective.
3. Discuss how replacing the Brier score by the asymmetric economic loss affects the fitted network and its predictions. In particular, discuss changes in the distribution of predicted default probabilities and the balance between errors on defaults and non-defaults. You should explain these differences rather than only report that one score is lower or higher.
4. Arrive at a clear conclusion about the effect of the asymmetric loss and about which model is preferable under each evaluation criterion.

You are not asked to explain how neural networks work in general. Focus on the choices you made and what you learned from the comparison between the two objectives.

Write your report in the provided L^AT_EX template, called `report.tex`. You are free to add packages of your choosing, but do not change the font, margins or other aspects of the template that determine how much text fits on the page. The report, returned as `report.pdf`, may not exceed **one page**. A list of references and an appendix may exceed the one-page limit. In the appendix, you may only list optimized hyperparameters, candidate values and figures; do not add additional discussion. Please enter your names and group number at the top of the template.

3 Group Registration

The assignment is supposed to be completed in teams of four. You should join your group in Canvas. The group registration deadline is stated on the Canvas assignment page. You are only guaranteed to work with group members of your choice if your group consists of four members. If groups have fewer than four students after the registration deadline, we may break up or rearrange them into groups of four.

4 Assignment Submission

The assignment submission consists of the following three deliverables:

1. `predictions.npy`, containing the two columns of probability predictions described above;
2. `report.pdf`, the report described in Section 2.2; and
3. the complete replication code used to train the two final neural networks and generate `predictions.npy`.

You will submit these deliverables via Canvas. After joining a group, it is sufficient if one group member submits the deliverables on behalf of the group. The deadline for providing all deliverables is **Friday, October 9, 2026, at 23:59 (Week 6)**. If you have questions about the assignment, its intended workflow or the submission format, please post them in the corresponding discussion forum on Canvas.

A Checklist

- ☐ I have read this document thoroughly and joined my group in Canvas.
- ☐ I trained two final neural networks with the same architecture and preprocessing, one using the Brier score and one using the asymmetric economic loss.
- ☐ I did not use `X_test.csv` for model selection, tuning, preprocessing decisions or stopping decisions.

- ☐ I did not use external data or attempt to recover hidden outcomes by matching the assignment data to another dataset.
- ☐ By the assignment deadline (October 9, 23:59), I submitted `predictions.npy`, `report.pdf`, and the replication code.
- ☐ I checked that `predictions.npy` has shape $(n_{\text{test}}, 2)$, is in the correct row and column order, and contains only finite probabilities in $[0, 1]$.
- ☐ I checked that the submitted replication code reproduces the submitted predictions.